Available online at www.sciencedirect.com

ScienceDirect

Procedia Computer Science 246 (2024) 576–585

Procedia Computer Science

www.elsevier.com/locate/procedia

28th International Conference on Knowledge-Based and Intelligent Information & Engineering Systems (KES 2024)

Smart Diagnosis of Active Systems

Gianfranco Lamperti[a,*], Xiangfu Zhao[b]

[a]*Department of Information Engineering, University of Brescia, 25123 Brescia, Italy*
[b]*School of Computer and Control Engineering, Yantai University, 264005 Yantai, China*

Abstract

Automated diagnosis has always been a challenging task to AI. When model-based diagnosis is adopted, a model of the system is required in order to generate a set of diagnoses based on a collection of observations, where a diagnosis is a set of faulty components or, more generally, a set of faults ascribed to components. An active system (AS) is an asynchronous, distributed discrete-event system, whose model consists of a topology (how components are connected to one another), and a communicating automaton for each component (the mode in which a component reacts to events). A problem afflicting all model-based approaches to diagnosis is a possibly large number of diagnoses explaining the observations, which may jeopardize the task of a diagnostician in charge of monitoring the system, owing to the cognitive overload raised by an overwhelming number of faulty scenarios to examine. This is exacerbated in critical application domains, where, under uncertain conditions, an artificial agent is supposed to perform recovery actions in real-time, even in the order of milliseconds, to possibly restore the system. To make diagnosis of ASs viable in critical, real-time application domains, a Smart Diagnosis Engine is presented, which is grounded on two heuristics: (1) if a diagnosis δ is a superset of a diagnosis δ', then δ is ignored (*minimality*); (2) if the cardinality (number of faults) of a diagnosis δ is lower than the cardinality of a diagnosis δ', then δ is generated before δ' (*sorting*). Consequently, the diagnosis output consists in a sequence of *minimal* diagnoses that are generated in ascending order by cardinality. As indicated by the experimental results, the overall improvement is twofold: most likely diagnoses are generated upfront, thereby supporting real-time recovery actions; also, the abductive search in the behavior space of the AS is reduced considerably, owing to the pruning of the trajectories that will not generate minimal diagnoses, thereby resulting in an impressive reduction in both memory allocation and processing time.

Peer-review under responsibility of the scientific committee of the 28th International Conference on Knowledge Based and Intelligent information and Engineering Systems

Keywords: model-based reasoning; real-time diagnosis; minimal diagnosis; active systems; discrete-event systems; finite automata; uncertainty

1. Introduction

Historically, automated diagnosis is one of the first problems being faced by AI. Initially and up to the mid eighties, diagnosis-oriented expert systems were developed based on rules provided by human experts in a specific application

* Gianfranco Lamperti. Tel.: +39-030-371-5491 ; fax: +39-030-380-014.
E-mail address: gianfranco.lamperti@unibs.it

1877-0509
Peer-review under responsibility of the scientific committee of the 28th International Conference on Knowledge Based and Intelligent information and Engineering Systems
10.1016/j.procs.2024.09.466

domain. This rule-based approach to diagnosis suffered from strict dependency upon the (varying) experience of humans, possibly leading to discrepancies raising from different schools of thought. To overcome that drawback, a novel approach to diagnosis was proposed thereafter by two seminal works published in an issue of the *Artificial Intelligence* journal in 1987 [27, 17]. That novel approach, which was initially conceived for static systems, such as combinational circuits [16, 12, 9], was coined *model-based diagnosis* [13]. The novelty of the approach was that no expertise of humans was required in terms of rules that map symptoms to diagnoses (possibly ranked by probabilities), but only a model of the *normal* behavior of the system considered. The nature of the diagnostic technique is *consistency-based*: given a collection of observations of the system (e.g., a set of input/output measures), the diagnosis engine generates a collection of *conflict sets*, each set including some components of the system (e.g., a variety of digital devices) where at least one component needs to be faulty, otherwise a logical inconsistency arises. Alternative diagnoses, called *candidates*, are eventually enumerated based on such conflicts, each candidate being a set of faulty components. Since the set of different diagnoses that conform with the conflict sets may be very large, only *minimal* diagnoses are elicited as candidates: a diagnosis is minimal iff it is not a superset of any other diagnosis. The rationale is that, assuming equal probabilities of faults, a minimal diagnosis is more likely than a non-minimal one. The advantage in reducing the set of candidates is twofold; on the one hand, the cognitive load of a diagnostician in charge of examining the candidates in order to possibly suggest recovery actions may be reduced to a manageable extent; on the other, the diagnosis engine becomes more efficient in both response time and memory allocation when non-minimal diagnoses can be ignored upfront, as the search space for minimal diagnoses is bound to shrink considerably.

When time-varying (dynamical) systems come into play (e.g., a nuclear power plant), the consistency-based approach conceived for static systems may be less than optimal, especially when a discrete-event model is adopted, namely when a *discrete-event system* (DES) is considered [6]. The model of a DES may be a Petri net, such as in [14, 5, 2, 7], or a network of communicating automata [4], like in [1, 8, 26, 10, 15, 18, 11, 24]. Model-based diagnosis of DESs stems from the seminal work by Sampath et al. [28, 29], where not only the normal, but also the faulty behavior of each component is specified. The availability of a complete model of the DES allows the diagnosis task to be *abduction-based* [25]: the behavior of the DES is reconstructed (abduced) based on a given temporal observation, namely a sequence of observations generated by a trajectory of the DES, a trajectory being a sequence of component transitions collectively changing the state of the DES. Specifically, the reconstructed behavior represents a set of trajectories of the DES that conform with the temporal observation. Each trajectory is associated with a diagnosis (candidate), which is typically the set of faulty events/transitions involved in the trajectory. Since only a (possibly tiny) fraction of the component transitions is observable, the actual trajectory of the DES producing the temporal observation is generally unknown; hence, several (typically many) candidates are generated. Diagnosis of DESs can be either *a posteriori* or *monitoring-based*. When a posteriori, the diagnosis engine is activated after the reception of a complete temporal observation, such as after the reaction of the protection apparatus of a power network subjected to a black out [19]. In monitoring-based diagnosis, instead, the diagnosis engine is expected to generate the candidates upon the reception of each newly-occurred observation, a task carried out when the DES being operated is required to be supervised in order to perform recovery actions in critical scenarios under stringent time constraints [22, 23, 20, 3].

The contribution of this paper is to present an a posteriori minimal-diagnosis technique for a class of asynchronous, distributed DESs called *active systems* (ASs). To our knowledge, only one approach to minimal diagnosis of DESs has been proposed in the literature [30], which formally rephrases the seminal work of Sampath et al. cited above in terms of minimal diagnosis and diagnosability. Hence, the diagnosis task in [30] is monitoring-based and, most importantly, requires the generation of a (minimal) diagnoser, an elegant data structure allowing the online diagnosis engine to perform in linear time. In practice, however, the generation of the diagnoser requires the generation of the complete behavior space of the DES, which is typically prohibitive in real application domains, owing to the exponential explosion of the number of states involved. This is why, in this paper, we present an a posteriori diagnosis technique for ASs that does not require any offline generation of a diagnoser. That Smart Diagnosis Engine is grounded on two heuristics, namely *minimality* and *sorting*: only minimal diagnoses are generated, with these minimal diagnoses (candidates) being sorted in ascending order by cardinality. Experimental results indicate that the overall improvement is twofold: most likely candidates are generated upfront, thereby supporting real-time recovery actions; furthermore, the search carried out by the diagnosis engine within the behavior space of the AS is shrunk, owing to the pruning of the trajectories that cannot generate minimal diagnoses, thereby reducing both memory allocation and processing time.

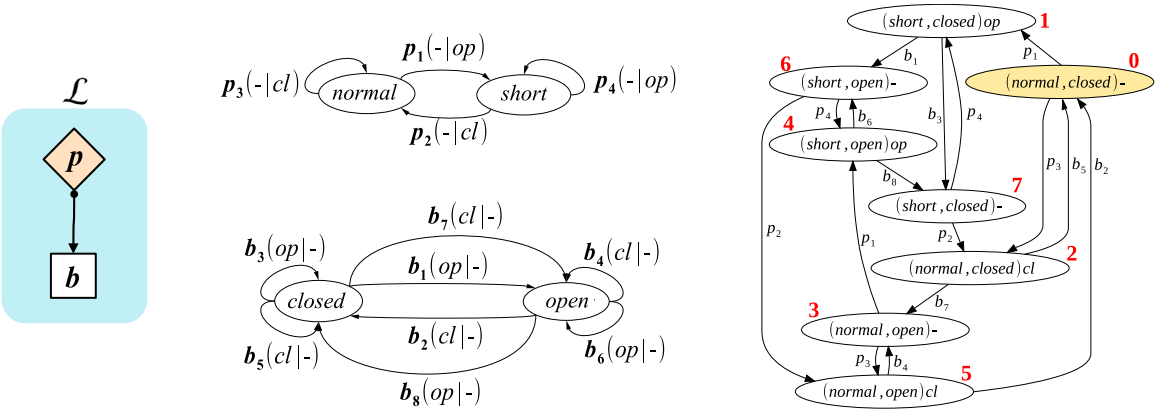

Figure 1: Left: AS $\mathcal{L}$, involving protection p and breaker b; center: models of p (top) and b (bottom); right: $Space(\mathcal{L})$.

2. Preliminaries

An AS is a distributed, asynchronous DES, whose topology is a network of *components* that are modeled as finite communicating automata. Components may be connected to other components via communication channels, called *links*. When the AS starts being operated, each component is in its initial state and links are empty: the AS is idle. The occurrence of an event outside the AS, such as a short circuit on a power transmission line, may trigger a state transition in a component, for instance, a protection device, which may generate events towards another component via a link, for example, a breaker required to open in order to isolate the line and, possibly, extinguish the short circuit. This results in a cascade of state transitions, called a *trajectory*, that moves the AS from its initial state, i.e., an array of the initial states of its components, to a new state, i.e., a new array of component states and a new array of link configurations, each configuration being characterized by a (possibly empty) event within the link. Trajectories of an AS $\mathcal{X}$ are confined to a *space*, which is formally represented as a finite automaton $Space(\mathcal{X}) = (\Sigma, X, \tau, x_0)$, where the alphabet Σ is the set of component transitions, X is the set of states, where a state is a pair (C, L), with C being the array of states of components and L being the array of the events within links, τ is the transition function mapping a state and a component transition into a new state, and x_0 is the initial state.

Example 1. Outlined on the left side of Figure 1 is an AS, called $\mathcal{L}$ (*line*), which is designed to protect a power transmission line from short circuits. It embodies two components, a protection p and a breaker b, and a link connecting p to b. A short circuit striking the line is perceived by the protection as a drop in voltage: if so, the protection commands the breaker to open in order to get the short circuit extinguished, in which case the protection commands the breaker to close in order to restore the electric power. The models (communicating automata) of both p (top) and b (bottom), which are displayed in the center of the figure, involve two states (represented as ellipses), with four and eight transitions, respectively (represented as arrows between states). Each transition is marked with its name, e.g. p_1, followed by a pair $(e|e')$, where e is the triggering event and e' is the possibly output event. An hyphen ('$-$') as input indicates an external event, e.g., a drop in voltage in $p_1(-|op)$, while an hyphen as output indicates no event, as in all transitions of b. The space of $\mathcal{L}$ is shown on the right side, where states are renamed $0 \ldots 7$, with 0 being the initial state. Each state is marked with a pair of the component states p and b, along with the possibly empty ('$-$') event within the link. A trajectory of $\mathcal{L}$ is, for instance, $T = [p_3, b_5, p_1, b_3, p_4, b_3, p_2, b_5]$. Owing to cycles, the space contains an infinite number of trajectories.

For diagnosis purposes, we specify both the *observability* and *abnormality* of $\mathcal{X}$ by means of a *mapping table*, namely $Map(\mathcal{X})$, which is based on two finite sets of labels: a set $\mathbf{O}$ of *observations*, and a set $\mathbf{F}$ of *faults*. Denoting with $\mathbf{T}$ the whole set of component transitions in $\mathcal{X}$, $Map(\mathcal{X})$ is a function mapping each component transition $t \in \mathbf{T}$ into a pair (o, f), where $o \in \mathbf{O} \cup \{\varepsilon\}$ and $f \in \mathbf{F} \cup \{\varepsilon\}$. $Map(\mathcal{X})$ can be represented as a set of triples (t, o, f), where each triple defines the observability and abnormality of component transition t, specifically: if $o \neq \varepsilon$, then t is *observable*, or else t is *unobservable*; likewise, if $f \neq \varepsilon$, then t is *faulty*, or else t is *normal*. Based on $Map(\mathcal{X})$, each trajectory

T of $\mathcal{X}$ is associated with a *temporal observation*. The temporal observation of T, namely $Obs(T)$, is a sequence of the observations associated with the component transitions in T: $[\, o \mid t \in T, (t, o, f) \in Map(\mathcal{X}), o \neq \varepsilon \,]$. A temporal observation O and a trajectory T are said to *conform* with each other iff $O = Obs(T)$. Note how O may conform with several (possibly infinite) trajectories, while T conforms with one temporal observation only. $Map(\mathcal{X})$ also associates T with a *diagnosis*. The diagnosis of T, namely $Dgn(T)$, is a set of faults associated with the component transitions in T: $\{\, f \mid t \in T, (t, o, f) \in Map(\mathcal{X}), f \neq \varepsilon \,\}$. The number of faults (cardinality) of δ is denoted $|\,\delta\,|$. A diagnosis δ is said to *explain* O iff $\delta = Dgn(T)$ and O conforms with T. The *diagnosis set* $\mathcal{D}$ of O is a set of the diagnoses of the trajectories of $\mathcal{X}$ conforming with O, namely: $\mathcal{D}(O) = \{\delta \mid \delta = Dgn(T), T \in Space(\mathcal{X}), O = Obs(T)\}$.

Example 2. Assuming $\mathbf{O} = \{prt, brk\}$ and $\mathbf{F} = \{f_1, f_2, f_3, f_4, f_5, f_6\}$, a mapping table for $\mathcal{L}$ is defined as $Map(\mathcal{L}) = \{(p_1, prt, \varepsilon), (p_2, prt, \varepsilon), (p_3, \varepsilon, f_1), (p_4, \varepsilon, f_2), (b_1, brk, \varepsilon), (b_2, brk, \varepsilon), (b_3, \varepsilon, f_3), (b_4, \varepsilon, f_4), (b_5, brk, \varepsilon), (b_6, brk, \varepsilon), (b_7, brk, f_5), (b_8, brk, f_6)\}$. Only one observation is defined for both the protection and the breaker, namely *prt* and *brk*, respectively. Thus, transition p_1 is observable and normal, p_3 is unobservable and faulty, and b_7 is both observable and faulty. Since the same observation is associated with several transitions, uncertainty remains in the abduction of the component transition based only on the observation occurred. With reference to the trajectory $T = [p_3, b_5, p_1, b_3, p_4, b_3, p_2, b_5]$ in Example 1, we have $Obs(T) = [brk, prt, prt, brk]$ and $Dgn(T) = \{f_1, f_2, f_3\}$. Based on $Space(\mathcal{L})$ in Figure 1, we can find that the diagnosis set of $O = [brk, prt, prt, brk]$, namely $\mathcal{D}(O)$, includes six diagnoses: $\{f_1, f_3\}$, $\{f_1, f_2, f_3\}$, $\{f_1, f_3, f_5\}$, $\{f_1, f_2, f_3, f_5\}$, $\{f_1, f_3, f_4, f_5\}$, and $\{f_1, f_2, f_3, f_4, f_5\}$.[1]

3. Smart Diagnosis

In this section, an efficient technique for the incremental generation of *minimal* diagnoses is presented, which is substantiated by a novel algorithm called Smart Diagnosis Engine (Algorithm 1). To this end, we first define the notions of a *minimal diagnosis* and a *candidate set* (Definition 1).

Definition 1. *A diagnosis in a diagnosis set $\mathcal{D}(O)$ is* minimal *if it is not a superset of any other diagnosis in $\mathcal{D}(O)$. The* candidate set *of O is the subset $\Delta(O) \subseteq \mathcal{D}(O)$ of all minimal diagnoses, each minimal diagnosis being called a* candidate *of O.*

Proposition 1. *If $\Delta(O)$ includes the empty candidate, then it does not include any other candidate, i.e. $\Delta(O) = \{\emptyset\}$.* (*Proof*) Any other candidate would include the empty candidate $\emptyset$ and, hence, it would not be minimal.

Corollary 1.1. *If $O = [\,]$, then $\Delta(O) = \{\emptyset\}$.* (*Proof*) O conforms with the empty trajectory $T = [\,]$, as $Obs(T) = [\,]$. Since $Dgn(T) = \emptyset$, the empty diagnosis $\emptyset$ is in $\Delta(O)$; hence, based on Proposition 1, $\Delta(O) = \{\emptyset\}$.

Definition 2. *A* sorting *of $\Delta(O) = \{\delta_1, \ldots, \delta_m\}$ is a sequence $[\delta'_1, \ldots, \delta'_m]$ where $\{\delta'_1, \ldots, \delta'_m\} = \{\delta_1, \ldots, \delta_m\}$ and, $\forall k \in [1\,..\,(m-1)]$, $|\,\delta'_k\,| \leq |\,\delta'_{k+1}\,|$.*

Example 3. With reference to $\mathcal{D}(O)$ in Example 2, the only minimal diagnosis is $\{f_1, f_3\}$, as it is included in all other diagnoses. Therefore, the candidate set of O is a singleton, namely $\Delta(O) = \{\{f_1, f_3\}\}$.

The Smart Diagnosis Engine algorithm exploits the notion of an *abduction item* (Definition 3) which, intuitively, represents the set of faults associated with a prefix of a trajectory conforming with a temporal observation.

Definition 3. *Let $O = [o_1, \ldots, o_n]$ be a temporal observation of $\mathcal{X}$ with set of faults $\mathbf{F}$. An* abduction item *of O is a triple (x, δ, i), where x is a state in $Space(\mathcal{X})$, δ is a set of faults in $\mathbf{F}$, and $i \in [0\,..\,n]$ is an* index *of O.*[2] *The* frontier *of an abduction item $\alpha = (x, \delta, i)$, $Front(\alpha)$, is a set of abduction items (x', δ', i') such that: $\langle x, t, x' \rangle$ is a transition in $Space(\mathcal{X})$, $(t, o, f) \in Map(\mathcal{X})$, if $f = \varepsilon$ then $\delta' = \delta$ else $\delta' = \delta \cup \{f\}$, and, if $o = \varepsilon$ then $i' = i$, else if $i < n$ then $i' = i + 1$.*

[1] As expected, $\mathcal{D}(O)$ includes $Dgn(T) = \{f_1, f_2, f_3\}$; yet, owing to uncertainty, five additional diagnoses are embodied in the diagnosis set.
[2] An index i is meant to indicate the prefix $[o_1, \ldots, o_i]$ of O matched so far, specifically, up to the i-th observation.

Algorithm 1: SMART DIAGNOSIS ENGINE

input : $O = [o_1, \ldots, o_n]$, a temporal observation of an AS $\mathcal{X}$ with initial state x_0
output : incremental generation of a sorting by cardinality of the candidate set $\Delta(O)$
variables : C: a candidate cardinality
$\mathcal{A}$: a set of abduction items
$\mathcal{A}_C$: a stack of abduction items where $|\delta| = C$
$\mathcal{A}_{C+1}$: a stack of abduction items where $|\delta| = C + 1$
Δ: a sequence of candidates

```
1  C ← 0,  A ← {(x0, ∅, 0)},  A_C ← ⌊(x0, ∅, 0)⌋,  A_{C+1} ← ⌊ ⌋,  Δ ← [ ]
2  repeat
3      repeat
4          Pop an abduction item α = (x, δ, i) from A_C
5          if α is not marked in A then
6              if i = n then
7                  Output δ and append it to Δ
8                  Mark in A every abduction item (x', δ', i') ∈ A_C where δ' = δ
9                  Mark in A every abduction item (x'', δ'', i'') ∈ A_{C+1} where δ'' ⊇ δ
10             else
11                 foreach abduction item α_f = (x_f, δ_f, i_f) ∈ Front(α) do
12                     if α_f ∉ A then
13                         Insert α_f into A
14                         if |δ_f| = C + 1 then
15                             if there is δ̄ ∈ Δ where δ̄ ⊆ δ_f then
16                                 Mark α_f in A
17                             else
18                                 Push α_f onto A_{C+1}
19                         else
20                             Push α_f onto A_C
21             Mark α in A
22     until A_C is empty
23     C ← C + 1
24     Swap A_C and A_{C+1}
25 until A_C is empty
```

Example 4. Considering the system $\mathcal{L}$ outlined in Figure 1, the relevant mapping table defined in Example 2, and the temporal observation $[brk, prt, prt, brk]$, an abduction item is $\alpha = (2, \{f_1\}, 0)$, where $2 = ((normal, closed), cl)$ is a state in $Space(\mathcal{L})$ (cf. right side of Figure 1). Hence, $Front(\alpha) = \{(3, \{f_1, f_5\}, 1), (0, \{f_1\}, 1)\}$, since, based on $Map(\mathcal{L})$, transition b_7 is both observable (brk) and faulty (f_5), while transition b_5 is observable (brk) and normal.

We are now ready to introduce the pseudocode of the SMART DIAGNOSIS ENGINE algorithm (Algorithm 1, lines 1–25), which takes as input a temporal observation O of an AS $\mathcal{X}$, with initial state x_0, and generates as output a sorting of the candidate set $\Delta(O)$ (cf. Definition 2). It makes use of a *set* $\mathcal{A}$ of abduction items, and two *stacks* of abduction items, namely $\mathcal{A}_C$, relevant to diagnoses with cardinality C, and $\mathcal{A}_{C+1}$, relevant to diagnoses with cardinality $C + 1$. Each newly-generated candidate is output and appended to Δ. Initially (line 1), the cardinality $C = 0$ refers to the empty candidate, while both $\mathcal{A}$ and $\mathcal{A}_C$ contain just the initial abduction item $(x_0, \emptyset, 0)$, with $\mathcal{A}_{C+1}$ being empty. The actual processing is performed in a loop (lines 2–25) that continues until $\mathcal{A}_C$ is empty, meaning that no further abduction item can be generated. At each iteration, a nested loop is repeated (lines 3–22) until $\mathcal{A}_C$ is empty, after which C is

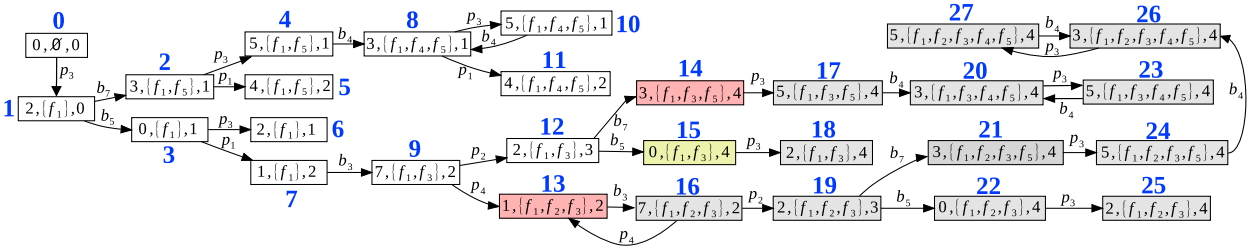

Figure 2: Graph of the abduction items for the temporal observation $O = [brk, prt, prt, brk]$ (cf. Example 5).

incremented, with $\mathcal{A}_C$ and $\mathcal{A}_{C+1}$ being swapped (lines 23 and 24). The idea is to process in $\mathcal{A}_C$ all the abduction items relevant to the current cardinality C, with the items with cardinality $C + 1$ being momentarily put into $\mathcal{A}_{C+1}$, which, after the swap, will become the new current abduction items relevant to the incremented cardinality (line 23). At each iteration of the inner loop, an item $\alpha = (x, \delta, i)$ is popped from $\mathcal{A}_C$ (line 4). Only if α has not been processed already (not marked in $\mathcal{A}$), is α considered (lines 6–21). If all observations in O have been matched ($i = n$), then δ is a new candidate (with cardinality C), which is thus output and appended to Δ (line 7). Then, in order to avoid the generation of non-minimal candidates, every item in $\mathcal{A}$ with diagnosis δ is marked, as is every item in $\mathcal{A}_{C+1}$ with a diagnosis including δ (lines 8 and 9). This prevents the algorithm from processing items with a (non-minimal) diagnosis that is either equal to or a superset of δ. Intuitively, the search space rooted in an abduction item is pruned whenever the corresponding processing cannot generate any new candidate. If, instead, $i < n$ (line 10), each abduction item $\alpha_f = (x_f, \delta_f, i_f)$ in the frontier of α is considered in an inner loop (lines 11–20), provided that α_f has not yet been processed (not in $\mathcal{A}$, line 12). If so, after inserting α_f into $\mathcal{A}$ (line 13), two cases are possible: either the cardinality of δ_f equals $C + 1$ (meaning that a new fault is involved in the component transition from x to x_f), or $|\delta_f| = |\delta|$. In the first case, if δ_f is not minimal (line 15), then α_f is marked (line 16), else it is pushed onto $\mathcal{A}_{C+1}$ (line 18) in order to be processed at the next iteration of the outer loop.

Example 5. With reference to the temporal observation $O = [brk, prt, prt, brk]$ of $\mathcal{L}$ considered in Example 2, shown in Figure 2 is the graph of the relevant abduction items (renamed $0\,..\,27$), where 0 is the item initially inserted into $\mathcal{A}$ and $\mathcal{A}_C$ in line 1. In the graph, an item $\alpha = (x, \delta, i)$ is connected with an item $\alpha' = (x', \delta', i')$ in its frontier by an arc $\langle \alpha, t, \alpha' \rangle$, where $\langle x, t, x' \rangle$ is a transition in $Space(\mathcal{L})$. Note that the graph includes all possible abduction items, not only those actually generated by Smart Diagnosis Engine.[3] The outer loop of the algorithm (lines 2–25) is expected to behave as follows. With $C = 0$, we have $\mathcal{A}_C = \lfloor 0 \rfloor$ and $\mathcal{A}_{C+1} = \lfloor 1 \rfloor$. With $C = 1$, the items processed in $\mathcal{A}_C$ are $1, 3, 6, 7$, while $\mathcal{A}_{C+1} = \lfloor 2, 9 \rfloor$. With $C = 2$, the items processed in $\mathcal{A}_C$ are $2, 4, 5, 9, 12, 15$, while $\mathcal{A}_{C+1} = \lfloor 8, 13, 14 \rfloor$. Since, in state 15, the observation index is 4 (fulfillment of condition in line 6), $\delta = \{f_1, f_3\}$ is a candidate. Based on line 8, the items in $\mathcal{A}_C$ with $\delta' = \{f_1, f_3\}$ need to be marked, but such items are missing in $\mathcal{A}_C$ at this point because, when processing item 15, all its predecessors, in particular items 9 and 12, have been processed (and marked) already. Based on line 9, instead, the items in $\mathcal{A}_{C+1}$ such that $\delta'' \supseteq \{f_1, f_3\}$ are 13 and 14, which are therefore marked. Note that, in line 21, item 15 is marked without having generated its frontier, hence item 18 will not be generated. Likewise, all successive items of 13 and 14 will not be generated also because both of them have been marked in line 9. With $C = 3$, the items processed in $\mathcal{A}_C$ are $8, 10, 11, 13, 14$. Since items 13 and 14 are marked and the processing of 10 and 11 does not create any new item, when all items in $\mathcal{A}_C$ have been processed ($\mathcal{A}_C$ is empty), $\mathcal{A}_{C+1}$ is empty too. Thus, after the swap in line 24, $\mathcal{A}_C$ is still empty, and the algorithm terminates. In the end, we have $\Delta(O) = \{\{f_1, f_3\}\}$, the same result anticipated in Example 3, with only 16 items (out of 28) being actually generated, where the diagnoses in the missing items $(16\,..\,27)$ are all supersets of $\{f_1, f_3\}$, in other words, they are not minimal diagnoses, as expected.

Before showing the experimental results in Section 4, we prove the correctness of Smart Diagnosis Engine, as claimed in Proposition 2.

Proposition 2. *Algorithm* Smart Diagnosis Engine *is correct.*

[3] A non-smart diagnosis engine, by contrast, will somewhat generate all these items in order to find all the six diagnoses in $\mathcal{D}(O)$ (cf. Example 2).

Proof (*sketch*). The proof is grounded on Lemmas 2.1–2.6.

Lemma 2.1. *The algorithm terminates.* (*Proof*) The set of abduction items $\alpha = (x, \delta, i)$ is finite because: the set of states in $Space(\mathcal{X})$ is finite, the set of possible diagnoses is bounded by $2^{\mathbf{F}}$, where $\mathbf{F}$ is the set of faults in $Map(\mathcal{X})$, and $i \leq n$, where n is the number of observations in $\mathcal{O}$. When α is processed, it is marked and neither processed again nor pushed onto the stacks. Hence, since the set of abduction items is finite, assuming that the algorithm does not terminate implies that, sooner or later, α will be processed again, a contradiction.

Lemma 2.2. *If $\delta \in \Delta$, then $\delta \in \mathcal{D}(\mathcal{O})$.* (*Proof*) Given $\alpha = (x, \delta, i)$, δ is appended to Δ in line 7 when $i = n$, i.e., when all observations in $\mathcal{O}$ are matched. This means that, based on Definition 3, there is a trajectory T in $Space(\mathcal{X})$ such that $\mathcal{O}$ conforms with T and $Dgn(T) = \delta$, as δ involves all faults in T; hence, $\delta \in \mathcal{D}(\mathcal{O})$.

Lemma 2.3. *Δ does not include duplicated diagnoses.* (*Proof*) When a diagnosis δ is appended to Δ in line 7, the relevant abduction item $\alpha = (x, \delta, i)$ is unmarked in $\mathcal{A}$ (condition in line 5). This means that all the other abduction items (x', δ, i') processed so far in $\mathcal{A}_C$ have $i' \neq n$, hence, δ is not in Δ already, or else, based on line 8, α would have been marked. In other words, Δ cannot include duplicated diagnoses.

Lemma 2.4. *If $\delta \in \Delta$, then $\delta \in \Delta(\mathcal{O})$ (soundness).* (*Proof*) Based on Lemma 2.2, $\delta \in \mathcal{D}(\mathcal{O})$. To show that δ is also minimal, consider the mode in which abduction items are marked. On the one hand, when a diagnosis δ is appended to Δ in line 7, all other abduction items (x', δ, i') in $\mathcal{A}_C$ are marked (line 8), as are the abduction items (x'', δ'', i'') in $\mathcal{A}_{C+1}$ where $\delta'' \supseteq \delta$ (line 9). On the other, if an abduction item $\alpha_f = (x_f, \delta_f, i_f) \in Front(\alpha)$ is such that $|\delta_f| = C + 1$ and there is $\bar{\delta} \in \Delta$ such that $\bar{\delta} \subseteq \delta_f$, then α_f is marked (lines 14–16). Based on condition in line 5, once marked, these abduction items cannot be processed and, therefore, no other diagnosis that contains δ can be generated. Hence, the diagnosis δ inserted into Δ is always minimal, in other words, $\delta \in \Delta(\mathcal{O})$.

Lemma 2.5. *If $\delta \in \Delta(\mathcal{O})$, then $\delta \in \Delta$ (completeness).* (*Proof*) From Definition 1, $\delta = Dgn(T)$, where $\mathcal{O}$ conforms with T; also, δ is minimal, i.e., it does not contain any other diagnosis $\delta' = Dgn(T')$, with $\mathcal{O}$ conforming with T'. In other words, T is a trajectory in $Space(\mathcal{X})$ that generates $\mathcal{O}$. On the other hand, starting from the initial abduction item $(x_0, \emptyset, 0)$, Smart Diagnosis Engine virtually traverses T by generating all successive abduction items such that, based on Definition 3, the sequence of states involved in the items is exactly the sequence of states involved in T: this means that, in the abduction item (x, δ, n) in line 7, $\delta = Dgn(T)$, with $\mathcal{O}$ conforming with T. Since δ is minimal, none of the abduction items generated is marked before the last item; therefore, all the items are actually generated, hence $\delta \in \Delta$.

Lemma 2.6. *If δ is followed by δ' in Δ, then $|\delta| \leq |\delta'|$.* (*Proof*) Smart Diagnosis Engine operates based on two *repeat* loops, where the inner loop (lines 3–22), detects all the candidates with cardinality C, where C is incremented at each iteration of the outer loop (line 23). This means that candidates (if any) are generated in ascending order and, hence, if $\Delta = [\ldots, \delta, \delta', \ldots]$, either $|\delta| = |\delta'|$ when they are generated in the same iteration, or $|\delta| < |\delta'|$, when δ' is generated in a successive iteration: in any case, $|\delta| \leq |\delta'|$.

In the end, based on Lemma 2.4 (soundness) and Lemma 2.5 (completeness), the set of diagnoses in Δ equals $\Delta(\mathcal{O})$. Moreover, according to Lemma 2.6, Δ is sorted in ascending order by cardinality of the candidates, in other words, based on Definition 2, Δ is a sorting of $\Delta(\mathcal{O})$. Hence, the algorithm generates incrementally a sorting of $\Delta(\mathcal{O})$. □

4. Experimentation

The diagnostic technique presented in this paper has been implemented and tested on a machine with CPU Intel i7-13620H @ 4.700GHz, 32GB of RAM, under the *Linux* operating system (*Kubuntu* 22.04.3 LTS). For comparison purposes, besides Smart Diagnosis Engine, also a Naive Diagnosis Engine has been developed, which generates the whole diagnosis set $\mathcal{D}(\mathcal{O})$, without any particular order. A special-purpose language for the specification of diagnosis problems has been designed and implemented in the *C* programming language, with the help of the *Flex* and *Bison* libraries (GNU distribution) for parsing the language. A diagnosis problem relevant to an AS is defined as a set of models (automata), a set of components, a set of links, a mapping table, and a temporal observation.

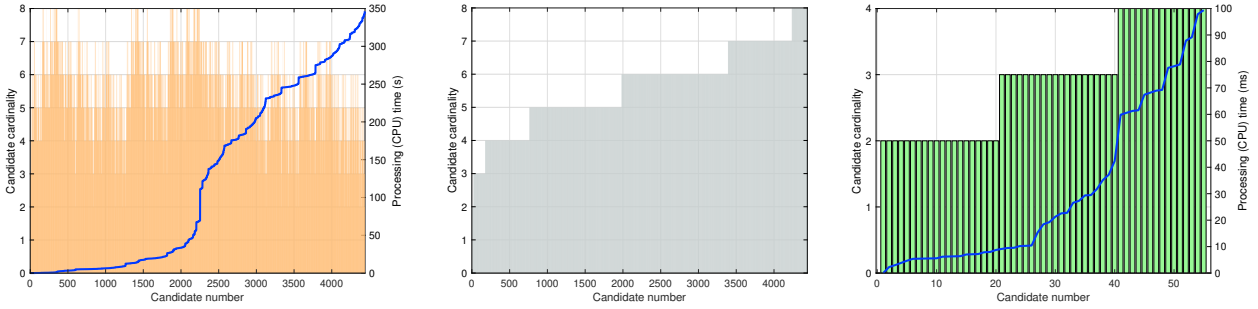

Figure 3: (*Scenario 1*) Naive Diagnosis Engine (left): 9348393 abduction items processed, with 4440 diagnoses generated in 351.81 seconds of total CPU time; and Smart Diagnosis Engine (right): 341710 abduction items processed, with 55 candidates generated in 0.63 seconds of total CPU time.

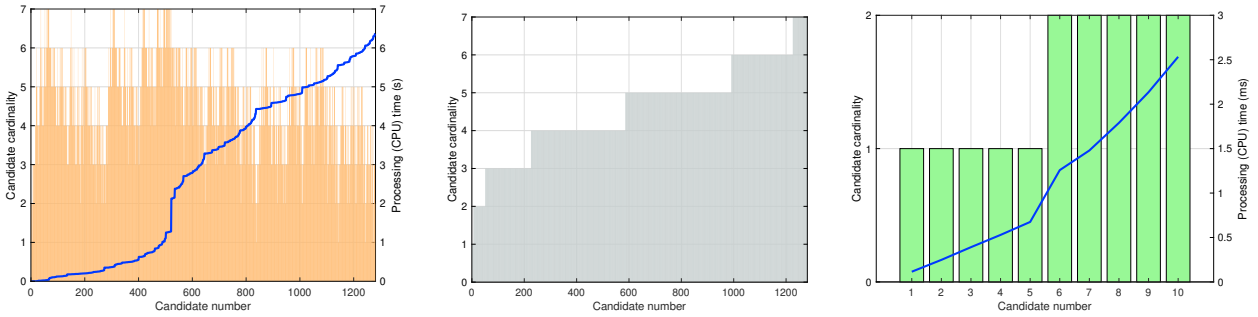

Figure 4: (*Scenario 2*) Naive Diagnosis Engine (left): 1021923 abduction items processed, with 1280 diagnoses generated in 6.50 seconds of total CPU time; and Smart Diagnosis Engine (right): 24613 abduction items processed, with 10 candidates generated in 0.03 seconds of total CPU time.

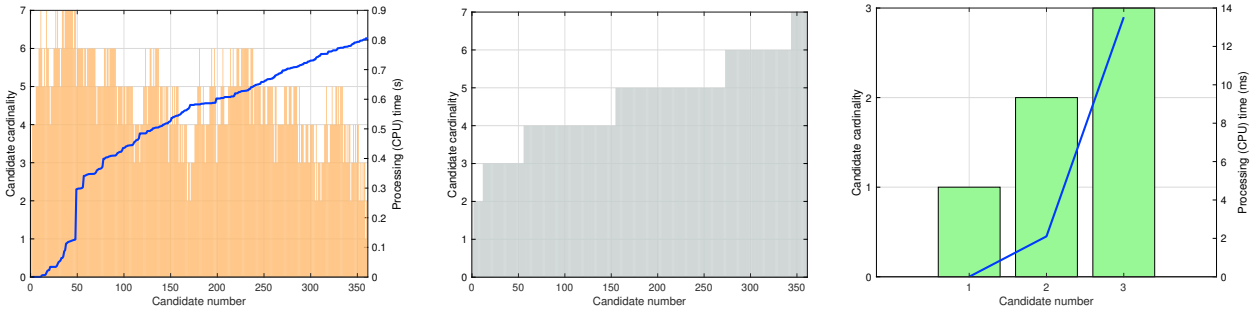

Figure 5: (*Scenario 3*) Naive Diagnosis Engine (left): 241943 abduction items processed, with 360 diagnoses generated in 0.80 seconds of total CPU time; and Smart Diagnosis Engine (right): 97122 abduction items processed, with 3 candidates generated, with 0.22 seconds of total CPU time.

Hereafter, we present the results of three scenarios conducted on an AS involving ten components, with varying mapping table and temporal observation. Outlined in Figure 3 are the results of the first scenario. The (orange) bar graph on the left represents the results of the naive engine, with the horizontal axis being labeled with the number of the candidate generated (from the first to the last), and the left-vertical axis indicating the cardinality of the candidate. The processing time (in seconds) spent up to the generation of the current candidate is indicated in the right-vertical axis and actually visualized as a line on the graph. The (gray) bar graph on the center outlines the same set of candidates shown on the left, which are sorted in ascending order by cardinality, from 2 to 8.[4] The results (for the same diagnosis problem) of the smart engine are shown in the (green) bar graph on the right, with axes having the same meaning as in the graph on the left (with processing time in milliseconds). The naive engine generates 4440 diagnoses (without any

[4] This sorting is not generated by the naive engine: it is just the result of (manual) postprocessing after the completion of the diagnosis task.

order) in almost six minutes of CPU time, with almost ten million abduction items being processed. The smart engine, instead, generates 55 candidates of increasing cardinality (2, 3, and 4) in 0.63 seconds, with 341710 abduction items. The fact that the completion of the candidates takes about 99 milliseconds (significantly less than the total CPU time) indicates that the engine keeps processing additional abduction items without generating new candidates.

The results of the second scenario are shown in Figure 4. The naive engine generates 1280 diagnoses in 6.5 seconds, with about one million abduction items. The smart engine, generates 10 candidates of increasing cardinality (1 and 2) in 0.03 seconds, with 24613 abduction items.

Finally, the results of the third scenario are shown in Figure 5. The naive engine generates 360 diagnoses in 0.8 seconds, with 241943 abduction items. The smart engine, generates only 3 candidates of increasing cardinality (1,2 and 3) in 0.22 seconds, with 97122 abduction items.

These figures suggest that, compared with Naive Diagnosis Engine, the advantages of Smart Diagnosis Engine are: (1) a significant reduction in the number of diagnoses (candidates), which is bound to alleviate the task of the diagnostician; (2) the generation of most likely candidates (with low cardinality) upfront, which allows the diagnostician to focus immediately on them before considering less probable candidates; and (3) a much faster response time, which allows a (possibly artificial) recovery agent to react in real-time in critical scenarios. Considering, for instance, the first scenario, 98.76% of the diagnoses are ignored by the smart engine, with 99.82% of processing time saved. Also, the naive engine takes 343 seconds to complete the generation of the 20 diagnoses with minimal cardinality (2 faults), while the smart engine takes only 8 milliseconds.

It is worth considering the results of a further experiment in which the naive engine generates 672 diagnoses in 124.17 seconds, with 4666718 abduction items, while the smart engine generates only the empty candidate (cf. Proposition 1) in 155μs, with 78 abduction items. Remarkably, the empty diagnosis is generated by the naive engine in 17.18 seconds (in position 433).

5. Conclusion

A major problem affecting all approaches to model-based diagnosis consists in a large number of candidates, which is bound to disorient a diagnostician in charge of examining all possible faulty scenarios, one for each candidate. Diagnosis of DESs is no exception: that problem is even more exacerbated when a diagnosis is expected under stringent time constraints, in order to be analyzed by an artificial agent designed to perform recovery actions autonomously and in real-time, in order to prevent dangerous, possibly catastrophic consequences. To mitigate this problem, on the one hand, a good heuristics is to avoid generating less likely diagnoses, specifically, non-minimal diagnoses. On the other, even if the set of candidates includes minimal diagnoses only, most likely candidates are those with low cardinality: the fewer the faults involved in a diagnosis, the more likely the diagnosis. Hence, it is also essential generating the candidates (minimal diagnoses) in ascending order by cardinality, so that most likely candidates are provided and examined upfront, before considering less likely candidates. In principle, both the elicitation of minimal diagnoses and their sorting by cardinality might be postponed after the generation of the *whole* set of diagnoses: the result in terms of (sorted) candidate set would be the same, but, remarkably, no improvement in performance of the diagnosis engine would be achieved. The Smart Diagnosis Engine proposed in this paper, instead, is capable of generating the ordered set of minimal diagnoses *upfront*, by pruning the search space of useless trajectories, thereby resulting in an impressive reduction of both memory allocation and time response, as testified by the experimental results. In the future, the approach can be extended to monitoring-based as well as to sequence-oriented diagnosis of DESs [21].

Acknowledgements

This work was supported by the EU NEXTGENERATIONEU program within the PNRR Future Artificial Intelligence – FAIR project (PE0000013, CUP H23C22000860006), Objective 10: Abstract Argumentation for Knowledge Representation and Reasoning, specifically by the *Argumentation for Informed Decisions with Applications to Energy Consumption in Computing – AIDECC* project (CUP D53C24000530001), and by the National Natural Science Foundation of China (grant number 61972360).

References

[1] Baroni, P., Lamperti, G., Pogliano, P., Zanella, M., 1999. Diagnosis of large active systems. Artificial Intelligence 110, 135–183. doi:10.1016/S0004-3702(99)00019-3.

[2] Basile, F., 2014. Overview of fault diagnosis methods based on Petri net models, in: Proceedings of the 2014 European Control Conference, ECC 2014, pp. 2636–2642. doi:10.1109/ECC.2014.6862631.

[3] Bertoglio, N., Lamperti, G., Zanella, M., Zhao, X., 2020. Temporal-fault diagnosis for critical-decision making in discrete-event systems, in: Cristani, M., Toro, C., Zanni-Merk, C., Howlett, R., Jain, L. (Eds.), Knowledge-Based and Intelligent Information & Engineering Systems: Proceedings of the 24th International Conference KES2020. Elsevier. volume 176 of *Procedia Computer Science*, pp. 521–530. doi:10.1016/j.procs.2020.08.054.

[4] Brand, D., Zafiropulo, P., 1983. On communicating finite-state machines. Journal of the ACM 30, 323–342. doi:10.1145/322374.322380.

[5] Cabasino, M.P., Giua, A., Seatzu, C., 2010. Fault detection for discrete event systems using Petri nets with unobservable transitions. Automatica 46, 1531–1539.

[6] Cassandras, C., Lafortune, S., 2008. Introduction to Discrete Event Systems. second ed., Springer, New York.

[7] Cong, X., Fanti, M., Mangini, A., Li, Z., 2018. Decentralized diagnosis by Petri nets and integer linear programming. IEEE Transactions on Systems, Man, and Cybernetics: Systems 48, 1689–1700.

[8] Debouk, R., Lafortune, S., Teneketzis, D., 2000. Coordinated decentralized protocols for failure diagnosis of discrete-event systems. Journal of Discrete Event Dynamic Systems: Theory and Applications 10, 33–86.

[9] El Fattah, Y., Dechter, R., 1995. Diagnosing tree-decomposable circuits, in: Fourteenth International Joint Conference on Artificial Intelligence (IJCAI 1995), Montreal, Quebec. pp. 1742–1748.

[10] Grastien, A., Cordier, M., Largouët, C., 2005. Incremental diagnosis of discrete-event systems, in: Nineteenth International Joint Conference on Artificial Intelligence (IJCAI 2005), Edinburgh, UK. pp. 1564–1565.

[11] Grastien, A., Haslum, P., Thiébaux, S., 2012. Conflict-based diagnosis of discrete event systems: theory and practice, in: Thirteenth International Conference on Knowledge Representation and Reasoning (KR 2012), Association for the Advancement of Artificial Intelligence, Rome, Italy. pp. 489–499.

[12] Hamscher, W., 1991. Modeling digital circuits for troubleshooting. Artificial Intelligence 51, 223–271.

[13] Hamscher, W., Console, L., de Kleer, J. (Eds.), 1992. Readings in Model-Based Diagnosis. Morgan Kaufmann, San Mateo, CA.

[14] Jiroveanu, G., Boel, R., Bordbar, B., 2008. On-line monitoring of large Petri net models under partial observation. Journal of Discrete Event Dynamic Systems 18, 323–354.

[15] Kan John, P., Grastien, A., 2008. Local consistency and junction tree for diagnosis of discrete-event systems, in: Eighteenth European Conference on Artificial Intelligence (ECAI 2008), IOS Press, Amsterdam, Patras, Greece. pp. 209–213.

[16] de Kleer, J., 1984. How circuits work. Artificial Intelligence 24, 205–280.

[17] de Kleer, J., Williams, B., 1987. Diagnosing multiple faults. Artificial Intelligence 32, 97–130.

[18] Kwong, R., Yonge-Mallo, D., 2011. Fault diagnosis in discrete-event systems: incomplete models and learning. IEEE Transactions on Systems, Man, and Cybernetics – Part B: Cybernetics 41, 118–130.

[19] Lamperti, G., Pogliano, P., 1997. Event-based reasoning for short circuit diagnosis in power transmission networks, in: Pollack, M. (Ed.), Fifteenth International Joint Conference on Artificial Intelligence (IJCAI 1997). Morgan Kaufmann Publishers, San Francisco, CA, pp. 446–451.

[20] Lamperti, G., Quarenghi, G., 2016. Intelligent monitoring of complex discrete-event systems, in: Czarnowski, I., Caballero, A., Howlett, R., Jain, L. (Eds.), Intelligent Decision Technologies 2016. Springer International Publishing Switzerland. volume 56 of *Smart Innovation, Systems and Technologies*, pp. 215–229. doi:10.1007/978-3-319-39630-9_18.

[21] Lamperti, G., Trerotola, S., Zanella, M., Zhao, X., 2023. Sequence-oriented diagnosis of discrete-event systems. Journal of Artificial Intelligence Research 78, 69–141. doi:10.1613/jair.1.14630.

[22] Lamperti, G., Zanella, M., 2004. A bridged diagnostic method for the monitoring of polymorphic discrete-event systems. IEEE Transactions on Systems, Man, and Cybernetics – Part B: Cybernetics 34, 2222–2244.

[23] Lamperti, G., Zanella, M., 2011. Monitoring of active systems with stratified uncertain observations. IEEE Transactions on Systems, Man, and Cybernetics – Part A: Systems and Humans 41, 356–369. doi:10.1109/TSMCA.2010.2069096.

[24] Lamperti, G., Zanella, M., Zhao, X., 2018. Introduction to Diagnosis of Active Systems. Springer, Cham. doi:10.1007/978-3-319-92733-6.

[25] McIlraith, S., 1997. Explanatory diagnosis: conjecturing actions to explain observations, in: Eighth International Workshop on Principles of Diagnosis (DX 1997), Mont St. Michel, France.

[26] Pencolé, Y., Cordier, M., 2005. A formal framework for the decentralized diagnosis of large scale discrete event systems and its application to telecommunication networks. Artificial Intelligence 164, 121–170.

[27] Reiter, R., 1987. A theory of diagnosis from first principles. Artificial Intelligence 32, 57–95.

[28] Sampath, M., Sengupta, R., Lafortune, S., Sinnamohideen, K., Teneketzis, D., 1995. Diagnosability of discrete-event systems. IEEE Transactions on Automatic Control 40, 1555–1575.

[29] Sampath, M., Sengupta, R., Lafortune, S., Sinnamohideen, K., Teneketzis, D., 1996. Failure diagnosis using discrete-event models. IEEE Transactions on Control Systems Technology 4, 105–124.

[30] Zhao, X., Ouyang, D., Lamperti, G., Tong, X., 2020. Minimal diagnosis and diagnosability of discrete-event systems modeled by automata. Complexity 2020, 1–17. doi:10.1155/2020/4306261.